



Question 1 [25 marks]

Consider the following loop.

```
LOOP: ld x10, 0(x13)
      ld x11, 8(x13)
      add x12, x10, x11
      subi x13, x13, 16
      bnez x12, LOOP
```

Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, that the pipeline has full forwarding support, and that branches are resolved in the EX (as opposed to the ID) stage. Also assume that the hazard detection unit is in place.

- a) [10 marks] A notation that names the fields of the pipeline registers allows for a more precise notation of dependences. For example, "ID/EX.RegisterRs1" refers to the number of one register whose value is found in the pipeline register ID/EX; that is, the one from the first read port of the register file. The first part of the name, to the left of the period, is the name of the pipeline register; the second part is the name of the field in that register.

Using this notation, the two pairs of hazard conditions are:

Type 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1

Type 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2

Type 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1

Type 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

Classify the **data hazards** in the above RISC-V assembly code

Classification of Dependencies

1. add x12, x10, x11 **depends on** ld x10, 0(x13) **(Type 1b)**
2. add x12, x10, x11 **depends on** ld x11, 8(x13) **(Type 2a)**
3. bnez x12, LOOP **depends on** add x12, x10, x11 **(Type 2a)**

- b) [10 marks] Show a **pipeline execution diagram** for the first two iterations of this loop.

```
ld x10, 0(x13)
ld x11, 8(x13)
add x12, x10, x11
subi x13, x13, 16
bnez x12, LOOP
```

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
ld x10, 0(x13)	IF	ID	EXE	MEM	WB											
ld x11, 8(x13)		IF	ID	EXE	MEM	WB										
add x12, x10, x11 becomes NOP			IF	ID	EXE	MEM	WB									
add x12, x10, x11				IF	ID	EXE	MEM	WB								
subi x13, x13, 16					IF	ID	EXE	MEM	WB							
bnez x12, LOOP						IF	ID	EXE	MEM	WB						
ld x10, 0(x13)							IF	ID	EXE	MEM	WB					
ld x11, 8(x13)								IF	ID	EXE	MEM	WB				
add x12, x10, x11 becomes NOP									IF	ID	EXE	MEM	WB			
add x12, x10, x11										IF	ID	EXE	MEM	WB		
subi x13, x13, 16											IF	ID	EXE	MEM	WB	
bnez x12, LOOP												IF	ID	EXE	MEM	WB

- c) [5 marks] Assume that each pipeline stage takes exactly one cycle (100 ps). What is the **speed up (for two iterations of the code)** when hazard detection and forwarding unit is enabled compared to when forwarding and hazard detection unit are not enabled.

No. of cycles with forwarding unit enabled = 16

No. of cycles with forwarding unit not enables = 20

Speedup = $20/16 = 1.25$

Question 2 [25 marks]

Assume that individual stages of the datapath of a RISC-V processor have the following latencies.

IF	ID	EX	MEM	WB
250ps	350ps	150ps	300ps	200ps

Also assume the “Registers” unit is written at the beginning of the cycle and read at the end of a cycle. Therefore, an ID stage will return the results of a WB stage occurring during the same cycle:

a. [5 marks] For this processor:

i. How many cycles are required to implement/execute one instruction on this pipeline?

Answer: 5 cycles

ii. How many cycles are required to execute 17 instructions on this pipeline? Assume that no stall cycles occur during the execution of all instructions.

Answer: 21 cycles (Explanation: 5 cycles for the first instruction, and an additional cycle for each of the next 16 instructions)

iii. How much total time (in ps) is required to execute one entire instruction if the five stages were not pipelined?

Answer: $250\text{ps} + 350\text{ps} + 150\text{ps} + 300\text{ps} + 200\text{ps} = 1250\text{ps}$

iv. What is the delay of 1 cycle (in ps) when all the five stages are pipelined?

Answer: 350ps (latency of the stage with the largest latency)

v. What is the total time (in ps) required to execute one entire instruction if the five stages are pipelined?

Answer: 5 times 350ps = 1750ps

b. [8 marks] Consider the following instruction sequence:

```
ld  x11, 8(x22)
add x11, x11, x12
sd  x11, 0(x22)
add x13, x11, x11
```

Assume that x11 is initialized to 11, x12 is initialized to 12, and x22 is the base address of A, an array of doublewords: A[] = {9, 12, 5, 9, 10, 12, 20, 13, 10, 13}

i. Suppose you executed the code above on a version of the pipeline that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final value of register x13 be if the code above is executed as it is (i.e., without inserting any NOP instructions)?

Answer:

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
ld x11, 8(x22)	IF	ID	EX	MEM	WB			
add x11, x11, x12		IF	ID	EX	MEM	WB		
sd x11, 0(x22)			IF	ID	EX	MEM	WB	
add x13, x11, x11				IF	ID	EX	MEM	WB
x11	11	11	11	11	12	23	23	23
x12	12	12	12	12	12	12	12	12
x13	X	X	X	X	X	X	X	24

Explanation:

In CC5, x11 is updated by the WB stage of the ld instruction. It loads the value from the memory at the address (x22 + 8) which is the address of A[1]. The value of A[1] is 12, so x11 changes value from its initial value, 11, to the new value 12.

In CC6, x11 is updated by the WB stage of the first add instruction. At the time when this add instruction is in ID stage (CC3), the values of x11 and x12 are 11 and 12 respectively. So the add instruction updates x11 as follows:

$$x11 = x11 + x12 = 11 + 12 = 23$$

In CC8, x13 is updated by the WB stage of the last add instruction. At the time when this add instruction reads x11 in the ID stage (CC5), the value of x11 is 12 (written in the first half of the CC5 by the WB stage of the ld instruction, which will be read by the ID stage of add instruction in the second half of CC5). So the last add instruction updates x13 as follows:

$$x13 = x11 + x11 = 12 + 12 = 24$$

ii. Add NOP instructions to the code above so that it will run correctly on a pipeline that does not handle data hazards

Answer:

We need to insert stalls so that the ID stage of each instruction that is dependent on the destination register of a previous instruction, is aligned with/after the WB stage of the instruction it depends upon.

“..” in the following diagram symbolizes a stall.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12
Ld x11, 8(x22)	IF	ID	EX	MEM	WB							
add x11, x11, x12		..	..	IF	ID	EX	MEM	WB				
sd x11, 0(x22)					..	..	IF	ID	EX	MEM	WB	
add x13, x11, x11								IF	ID	EX	MEM	WB

So the new code sequence with nops inserted to handle the data hazards is as follows:

```
ld x11, 8(x22)
nop
nop
add x11, x11, x12
nop
nop
sd x11, 0(x22)
add x13, x11, x11
```

c. [12 marks] Now assume that the pipeline has all the necessary circuitry for handling data hazards:

i. How many stalls will be generated when the code above executes on this pipeline? Draw a multi-cycle pipeline diagram to illustrate what happens in the first 9 cycles.

Answer:

1 stall will be generated after the `ld` instruction due to a load-use data hazard.

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9
<code>ld x11, 8(x22)</code>	IF	ID	EX	MEM	WB				
<code>add becomes nop</code>		IF	ID	..	..	..			
<code>add x11, x11, x12</code>			IF	ID	EX	MEM	WB		
<code>sd x11, 0(x22)</code>				IF	ID	EX	MEM	WB	
<code>add x13, x11, x11</code>					IF	ID	EX	MEM	WB

Note: The forwarding from MEM/WB to MEM stage (that `sd` instruction depends upon) was not taught in class, but it doesn't have to be taught explicitly for a student to know that forwarding can handle this situation.

ii. In clock cycle 5, which individual instruction is being processed by each of the 5 stages of the pipeline?

Answer:

<code>add x13, x11, x11</code>	<code>sd x11, 0(x22)</code>	<code>add x11, x11, x12</code>	<code>bubble</code>	<code>ld x11, 8(x22)</code>
IF	ID	EX	MEM	WB

iii. What will be the value of “RegWrite” in each of the first 9 cycles of execution? Also provide the value of “WriteData” input of the “Registers” unit during the cycles when “RegWrite” is asserted.

Answer:

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9
ld x11, 8(x22)	IF	ID	EX	MEM	WB				
add becomes nop		IF	ID	..	..	..			
add x11, x11, x12			IF	ID	EX	MEM	WB		
sd x11, 0(x22)				IF	ID	EX	MEM	WB	
add x13, x11, x11					IF	ID	EX	MEM	WB
RegWrite	0	0	0	0	1	0	1	0	1
WriteData	X	X	X	X	12	X	24	X	48

Explanation:

RegWrite is only asserted in the WB stage of instructions that have a destination register.

In CC5, the WB stage of ld instruction writes the value of A[1] into x11 so “WriteData” input of “Registers” unit is the value of A[1] i.e. 12

In CC7, the WB stage of the first add instruction updates x11 using the correct values of source registers thanks to forwarding. So $x11 = x11 + x12 = 12 + 12 = 24$. Hence “WriteData” input of “Registers” unit is 24.

In CC9, the WB stage of the last add instruction updates x13 using the correct values of source registers thanks to forwarding. So $x13 = x11 + x11 = 24 + 24 = 48$. Hence “WriteData” input of “Registers” unit is 48.

Q3 [25 marks] Attempt all of the following parts:

a) [8 marks] Suppose we have a direct-mapped cache with 32 blocks, where each block can hold 4 32-bit words. The memory address space is 16 bits (64KB), and the cache is byte-addressable.

i. [4 marks] How many bits from the main memory address will be allocated for tag, index, and block offset?

Tag	Index	Block Offset
-----	-------	--------------

(Note: Block offset also includes byte offset in its least significant bits)

ii. [4 marks] Draw a diagram for the cache organization to provide a high-level view of the implementation.

Answer:

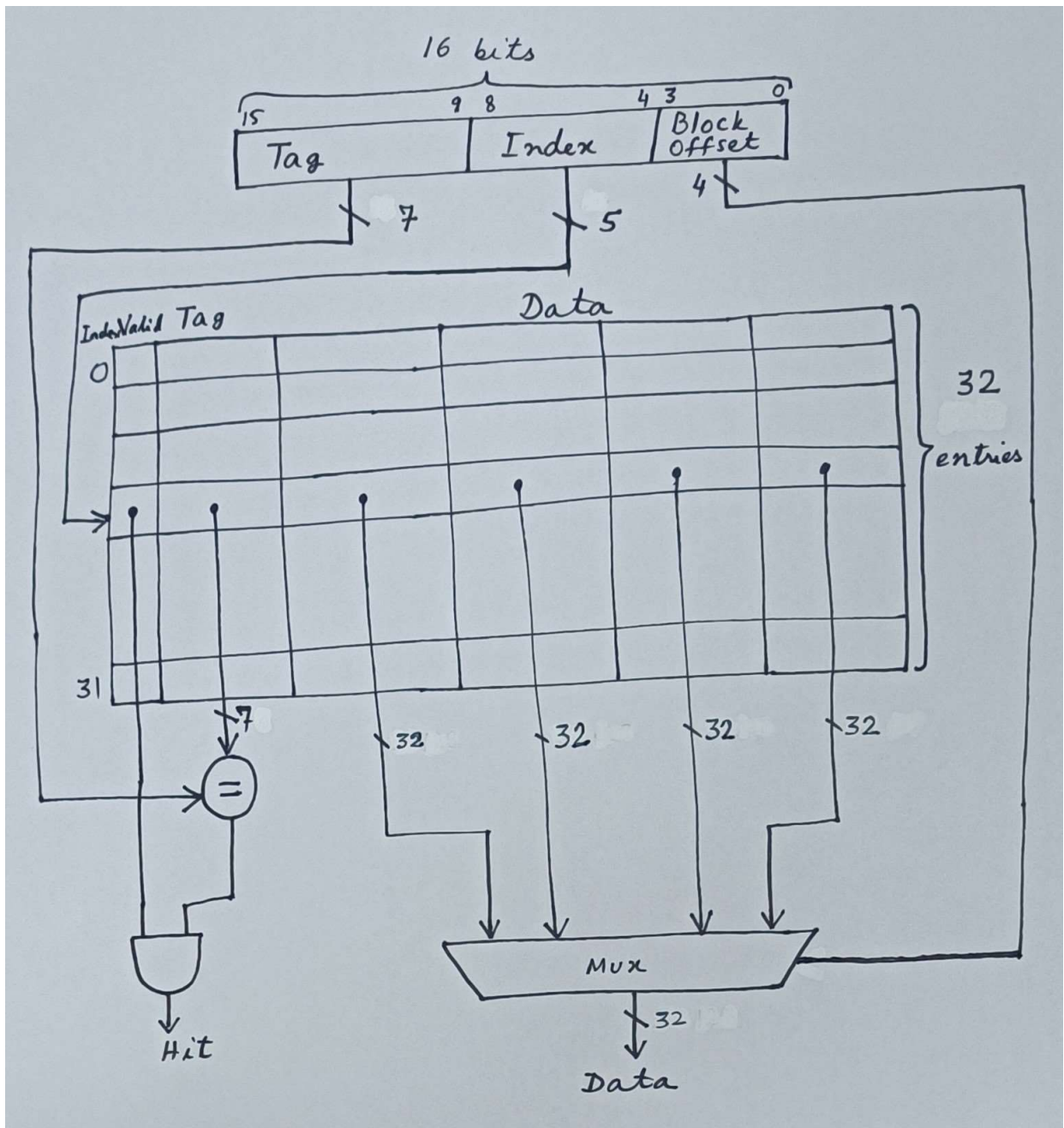
Tag = 7	Index = 5	Block Offset = 4
---------	-----------	------------------

- i. Each block holds 4 words = $4 \times 4 = 16$ bytes = 2^4 bytes, therefore, 4 bits are allocated for Block Offset.

32 blocks in Cache = 2^5 blocks in cache, therefore, 5 bits are allocated for index.

Total bits in memory address are 16, therefore, remaining $16 - 5 - 4 = 7$ bits are allocated for tag.

- ii.



- b) [8 marks] Suppose we have a 4-way set-associative cache with 64 blocks, where each block can hold 8 32-bit words. The memory address space is 32 bits (4GB), and the cache is byte-addressable.
- i. [4 marks] How many bits from the main memory address will be allocated for tag, index, and block offset?

Tag	Set	Block Offset
-----	-----	--------------

(Note: Block offset also includes byte offset in its least significant bits)

- ii. [4 marks] Draw a diagram for the cache organization to provide a high-level view of the implementation.

Answer:

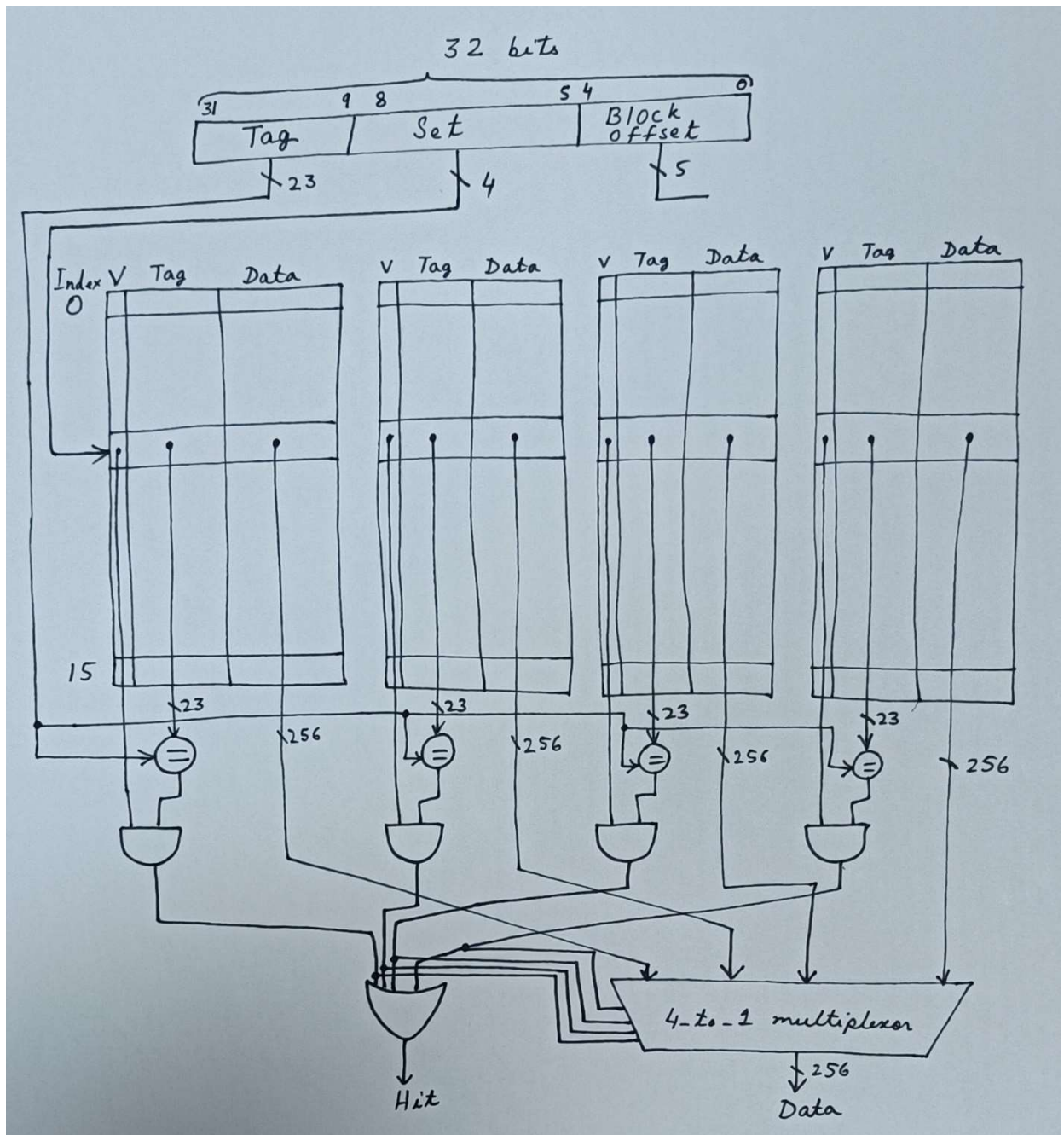
Tag = 23	Set = 4	Block Offset = 5
----------	---------	------------------

- i. Each block holds 8 words = $8 \times 4 = 32$ bytes = 2^5 bytes, therefore, 5 bits are allocated for Block Offset.

No. of sets in cache = 64 blocks in Cache / Associativity = $64 / 4 = 16$ sets = 2^4 sets in cache, therefore, 4 bits are allocated for index(set).

Total bits in memory address are 32, therefore, remaining $32 - 4 - 5 = 23$ bits are allocated for tag.

- ii.



- c) [4 marks] Assume the miss rate of an instruction cache is 2% and the miss rate of the data cache is 4%. Assume the frequency of all loads and stores is 30%.
- If a processor has a CPI of 2 without any memory stalls, and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed.

Answer:

The number of memory miss cycles for instructions in terms of the Instruction Count (I) is:

- Instruction miss cycles = $I \times 2\% \times 100 = 2.00 \times I$

As the frequency of all loads and stores is 30%, we can find the number of memory miss cycles for data references:

- Data miss cycles = $I \times 30\% \times 4\% \times 100 = 1.20 \times I$
- The total number of memory-stall cycles is $2.00 I + 1.20 I = 3.20 I$. This is more than three cycles of memory stall per instruction.
- Accordingly, the total CPI including memory stalls is $2.00 + 3.20 = 5.20$.

Since there is no change in instruction count or clock rate, the ratio of the CPU execution times is:

$$\frac{\text{CPU Time with stalls}}{\text{CPU time with perfect cache}} = \frac{I \times CPI_{\text{stall}} \times \text{Clock Cycle Time}}{I \times CPI_{\text{perfect}} \times \text{Clock Cycle Time}} = \frac{CPI_{\text{stall}}}{CPI_{\text{perfect}}} \\ = \frac{5.2}{2} = 2.6$$

The performance with the perfect cache is better by 2.6.

- d) [5 marks] Suppose we have a processor with a base CPI of 1, assuming all references hit in the primary cache, and a clock rate of 4 GHz. Assume a main memory access time of 100 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 3%.
- How much faster will the processor be if we add a secondary cache that has a 10-ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.75%?

Answer:

The miss penalty to main memory is:

$$\frac{100 \text{ ns}}{0.25 \text{ ns}} = 400 \text{ clock cycles}$$

The effective CPI with one level of caching is given by:

Total CPI = Base CPI + Memory-stall cycles per instruction

For the processor with one level of caching,

Total CPI = $1.0 + \text{Memory-stall cycles per instruction} = 1 + 3\% \times 400 = 13$

With two levels of caching, a miss in the primary (or first-level) cache can be satisfied either by the secondary cache or by main memory.

The miss penalty for an access to the second-level cache is:

$$\frac{10 \text{ ns}}{0.25 \text{ ns}} = 40 \text{ clock cycles}$$

If the miss is satisfied in the secondary cache, then this is the entire miss penalty. If the miss needs to go to main memory, then the total miss penalty is the sum of the secondary cache access time and the main memory access time.

Thus, for a two-level cache, total CPI is the sum of the stall cycles from both levels of cache and the base CPI:

$$\text{Total CPI} = 1.0 + \text{Primary stalls per instruction} + \text{Secondary stalls per instruction} = 1.0 + 3\% \times 40 + 0.75\% \times 400 = 1.0 + 1.2 + 3 = 5.2$$

Thus, the processor with the secondary cache is faster by:

$$\frac{13}{5.2} = 2.5$$

ALTERNATIVE SOLUTION:

Alternatively, we could have computed the stall cycles by summing the stall cycles of those references that hit in the secondary cache $((3\% - 0.75\%) \times 40 = 0.9)$. Those references that go to main memory, which must include the cost to access the secondary cache as well as the main memory access time, are $(0.75\% \times (40 + 400) = 3.3)$. The sum, $1.0 + 0.9 + 3.3$, is again 5.2.

Question 04 [25 marks]

Refer to the sequence of word addresses shown below.

0, 2, 3, 1, 0, 7, 8, 9, 10, 11, 2, 3, 1, 0, 8, 9, 11

- a) [12 marks] Consider a two-way set associative cache with a total of 4 one-word blocks. Assume a true LRU replacement policy. For each reference, fill in the following table.

Word Address	Hit/Miss	Set 0		Set 1	
0	Miss	M[0]			
2	Miss	M[0]	M[2]		
3	Miss	M[0]	M[2]	M[3]	
1	Miss	M[0]	M[2]	M[3]	M[1]
0	Hit	M[0]	M[2]	M[3]	M[1]
7	Miss	M[0]	M[2]	M[7]	M[1]
8	Miss	M[0]	M[8]	M[7]	M[1]
9	Miss	M[0]	M[8]	M[7]	M[9]
10	Miss	M[10]	M[8]	M[7]	M[9]
11	Miss	M[10]	M[8]	M[11]	M[9]
2	Miss	M[10]	M[2]	M[11]	M[9]

3	Miss	M[10]	M[2]	M[11]	M[3]
1	Miss	M[10]	M[2]	M[1]	M[3]
0	Miss	M[0]	M[2]	M[1]	M[3]
8	Miss	M[0]	M[8]	M[1]	M[3]
9	Miss	M[0]	M[8]	M[1]	M[9]
11	Miss	M[0]	M[8]	M[11]	M[9]

b) [13 marks] Consider a fully associative cache with a total of 8 one-word blocks.

Most Recently Used (MRU) is a cache replacement scheme which removes the most recently used items first. A MRU scheme is good in situations in which the older an item is, the more likely it is to be accessed.

Assume a true MRU replacement policy. For each reference given above, fill in the following table.

Word Address	Hit/Miss	Block 0	Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7
0	Miss	M[0]							
2	Miss	M[0]	M[2]						
3	Miss	M[0]	M[2]	M[3]					
1	Miss	M[0]	M[2]	M[3]	M[1]				
0	Hit	M[0]	M[2]	M[3]	M[1]				
7	Miss	M[0]	M[2]	M[3]	M[1]	M[7]			
8	Miss	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]		
9	Miss	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	
10	Miss	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[10]
11	Miss	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
2	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
3	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
1	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
0	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
8	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
9	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]
11	Hit	M[0]	M[2]	M[3]	M[1]	M[7]	M[8]	M[9]	M[11]